

K-means 1D (naive) com Paralelização Progressiva

Projeto PCD

Luís G. dos Santos¹

¹Instituto de Ciência e Tecnologia – Universidade Federal de São Paulo (UNIFESP)
Av. Cesare Mansueto Giulio Lattes, n° 1201
CEP: 12247-014 – Eugênio de Mello – SP – Brasil

Abstract. *The K-Means algorithm allows clustering samples from a given set into homogeneous groups. It is an efficient heuristic algorithm that can quickly find a local optimum, although clustering problems are classified as NP-hard. The objective of this work is to implement k-means in 1 dimension (points $X[i]$ and centroids $C[c]$), measure SSE (Sum of Squared Errors) and performance, and parallelize the core algorithm using three different approaches: a) OpenMP (CPU shared memory), b) CUDA (GPU), and c) MPI (distributed memory).*

Resumo. *O algoritmo K-Means permite agrupar amostras de um dado conjunto em grupos homogêneos. Ele é um algoritmos heurísticos eficientes que pode rapidamente encontrar um ótimo local, embora os problemas de "clusterização" sejam classificados como NP-hard. O objetivo deste trabalho é implementar o k-means em 1 dimensão (pontos $X[i]$ e centróides $C[c]$), medir SSE (Soma dos Erros Quadráticos) e desempenho, e paralelizar o núcleo do algoritmo usando três abordagens diferentes: a) OpenMP (CPU memória compartilhada), b) CUDA (GPU) e c) MPI (memória distribuída).*

1. Introdução

O algoritmo K-Means realiza uma operação de agrupamento ("clusterização") para mineração de dados, ou seja, permite agrupar amostras de um dado conjunto em grupos homogêneos. O método particiona um conjunto de n observações (pontos) em k grupos, onde cada ponto será associado ao grupo cuja média seja a mais próxima. A distância Euclidiana é geralmente a métrica adotada para medir a proximidade. A "clusterização" é um problema NP-hard, mas existem algoritmos heurísticos eficientes que podem rapidamente encontrar um ótimo local. Nesta implementação, a aplicação recebe como entrada as coordenadas (1D) de k centróides iniciais e um conjunto de dados. O K-Means realiza um processo iterativo, no qual os pontos são reagrupados de acordo com a menor distância Euclidiana entre eles e os centróides. Em seguida, o centróide de cada partição é recalculado tomando a média de todos os pontos da partição, e todo o procedimento é repetido até que nenhum centróide seja alterado e nenhum ponto seja atribuído a outro grupo. Ao final, o algoritmo retorna as coordenadas dos k centróides finais.

O objetivo deste trabalho é implementar o k-means em 1 dimensão (pontos $X[i]$ e centróides $C[c]$), medir SSE e desempenho, e paralelizar o núcleo do algoritmo em três etapas independentes: a) OpenMP (CPU memória compartilhada), b) CUDA (GPU) e c) MPI (memória distribuída).

2. Revisão Bibliográfica

A implementação de algoritmos em ambientes de processamento paralelo exige um tratamento cuidadoso da concorrência para garantir a corretude e a eficiência do programa. Esta seção aborda os conceitos fundamentais da programação concorrente, com base nos princípios descritos por Ben-Ari (2006).

2.1. O Modelo de Programação Concorrente

De acordo com [Ben-Ari 2006], um programa concorrente é definido como um conjunto de processos ou threads que são executados "ao mesmo tempo". Embora o paralelismo verdadeiro só ocorra em sistemas com múltiplos processadores, a intercalação (interleaving) da execução de instruções em um único processador apresenta os mesmos desafios lógicos.

2.2. O Problema da Seção Crítica

O desafio central da programação concorrente é gerenciar o acesso a recursos compartilhados. Uma **seção crítica** é definida como um bloco de código que acessa um ou mais desses recursos e que não deve ser executado por mais de um processo ou thread simultaneamente para evitar inconsistências nos dados [Ben-Ari 2006]. A implementação de um protocolo que garanta essa condição é a solução para o chamado **problema da seção crítica**.

Este problema manifesta-se de formas distintas nos paradigmas de programação utilizados neste trabalho:

- **Memória Compartilhada (OpenMP e CUDA):** Neste modelo, a seção crítica é evidente. Múltiplas threads (seja na CPU com OpenMP ou na GPU com CUDA) compartilham um mesmo espaço de endereçamento de memória. A tentativa de modificar simultaneamente uma variável global ou uma posição em um vetor compartilhado — como a soma e a contagem de pontos de um cluster na etapa de *update* do K-Means — constitui uma seção crítica que exige proteção.
- **Memória Distribuída (MPI):** No paradigma de troca de mensagens, o conceito de seção crítica para proteger uma variável compartilhada não existe, pois os processos, por definição, não compartilham memória. Cada processo opera em suas cópias privadas dos dados. No entanto, o conceito pode ser estendido para uma **seção crítica lógica**, onde apenas um processo por vez pode executar uma determinada ação, como escrever em um arquivo de saída compartilhado ou coordenar uma etapa do algoritmo. A sincronização, neste caso, não é feita com travas de memória, mas através de protocolos de comunicação explícita.

2.3. Condições de Corrida e a Necessidade de Atomicidade

Quando a proteção à seção crítica falha, o programa fica vulnerável a **condições de corrida**. Este fenômeno ocorre quando o resultado de uma computação depende da ordem não-determinística com que as threads são escalonadas pelo sistema [Ben-Ari 2006]. Uma instrução de alto nível, como `soma = soma + 1`, não é executada de forma **atômica**, sendo decomposta em múltiplas instruções de máquina (carregar, somar, armazenar). Se uma troca de contexto ocorrer entre essas instruções, uma atualização pode ser perdida, levando a resultados incorretos [Ben-Ari 2006].

- **Em OpenMP e CUDA:** O risco de condições de corrida é o principal desafio. No K-Means, se duas threads tentam incrementar o contador do mesmo cluster ao mesmo tempo, ambas podem ler o mesmo valor inicial, resultando em um incremento perdido. Para evitar isso, as atualizações devem ser atômicas. A solução em OpenMP pode ser de alto nível, como a cláusula `reduction` ou diretivas como `#pragma omp atomic`, enquanto em CUDA a solução idiomática é o uso de operações atômicas de hardware, como `atomicAdd()`.
- **Em MPI:** Condições de corrida em memória são impossíveis por design. Contudo, o conceito pode se manifestar na forma de **deadlocks de comunicação** ou resultados incorretos se a lógica do programa depender da ordem de chegada de mensagens, que é inerentemente não-determinística. Por exemplo, se um processo espera uma mensagem de A e depois de B, mas B envia primeiro, o programa deve ser robusto o suficiente para lidar com essa situação.

2.4. Soluções para a Exclusão Mútua

A garantia de que no máximo uma thread execute a seção crítica por vez é conhecida como **exclusão mútua** [Ben-Ari 2006]. A busca por algoritmos eficientes que garantam essa propriedade é um tópico central da ciência da computação. Soluções de software, como os algoritmos de Dekker e Peterson, provaram a viabilidade da exclusão mútua, mas à custa de uma **espera ocupada** (*busy-waiting*), onde a CPU é consumida em laços de espera [Ben-Ari 2006].

As abordagens modernas, relevantes para este trabalho, utilizam suporte de hardware e do sistema operacional para alcançar a exclusão mútua de forma mais eficiente.

- **OpenMP e CUDA:** Ambas as tecnologias se beneficiam diretamente das **instruções atômicas complexas** fornecidas pelo hardware, como *Compare-and-Swap* ou *Fetch-and-Add* [Ben-Ari 2006]. Em CUDA, essas instruções são expostas diretamente ao programador (ex: `atomicAdd()`). Em OpenMP, elas são a base sobre a qual abstrações de mais alto nível são construídas. A estratégia de **acumuladores por thread**, utilizada na implementação OpenMP do K-Means, é um padrão de projeto que contorna a necessidade de exclusão mútua em um recurso compartilhado, permitindo que cada thread trabalhe em dados privados e evitando a contenção.
- **MPI:** A exclusão mútua é alcançada através de **protocolos de comunicação**. Não se trata de travar uma variável, mas de coordenar o direito de agir. Por exemplo, pode-se implementar um algoritmo de "passagem de bastão" (*token passing*), onde apenas o processo que detém o "bastão" (uma mensagem especial) pode executar sua seção crítica lógica. Outra abordagem é ter um processo coordenador (geralmente o de rank 0) que recebe pedidos por mensagem e concede permissão de acesso, serializando as requisições.

3. Implementação Computacional

Primeiramente implementamos uma versão sequencial do algoritmo k-means em 1 dimensão ("naive"), para servir de base de comparação para as versões com paralelização. Além disso, todas as implementações usam como entradas dois arquivos CSV de 1 coluna, sem cabeçalho: a) dados.csv (com N valores) e b) centroides_iniciais.csv (com K valores). Há saídas no terminal (iterações, SSE (Sum of Squared Errors, ou Soma dos

Erros Quadráticos, em português) final, tempo total em ms) e em arquivo: a)assign.csv (N linhas, índice do cluster por ponto) e b)centroids.csv (K linhas com centróides finais).

Também foram construídos conjuntos de teste para serem usados por todas as implementações do algoritmos k-means 1D utilizados neste trabalho. O menor dos conjuntos (denominado "pequeno") tinha $N = 10^4 eK = 4$. O conjunto intermediário ("médio") tinha $N = 10^5 eK = 8$. Finalmente, o maior conjunto tinha $N = 10^6 eK = 16$. Inicialmente, todos eles continham dados gerados com mistura de faixas (ex.: perto de 0, 10, 20, 30) para facilitar a verificação visual do agrupamento dos pontos. Entretanto, posteriormente alteramos o programa de geração de dados para que eles fossem aleatoriamente distribuídos, para realmente testar a capacidade de agrupamento da solução.

3.1. Critérios de Validação e Corretude

A validação da corretude da implementação paralela é um passo crucial para garantir que os ganhos de desempenho não foram obtidos à custa de resultados incorretos. A versão sequencial, `kmeans_1d_naive.c`, foi utilizada como a linha de base (baseline) para a verdade dos resultados. A validação foi realizada em duas frentes:

- **Validação dos Resultados Finais:** Foi desenvolvido um script em Python para comparar os arquivos de saída gerados pelas versões sequencial e paralela. O script compara os arquivos `centroids.csv` para verificar se os centróides finais são numericamente equivalentes, e os arquivos `assign.csv` para analisar a consistência na atribuição de cada ponto de dado a um cluster.
- **Validação da Convergência Monotônica:** O algoritmo K-Means, por sua natureza, garante que o Erro Quadrático (SSE) nunca deve aumentar entre as iterações. Para verificar isso, uma verificação foi inserida no laço principal da implementação OpenMP. A cada iteração, o SSE atual é comparado com o da iteração anterior. Qualquer aumento no SSE indicaria a presença de uma condição de corrida que corrompeu o estado do algoritmo. Na implementação CUDA essa verificação foi adicionada dentro do loop de iteração principal, na função `kmeans_1d_cuda(querodanoHost/CPU)`.

3.2. Implementação usando OpenMP

O modelo de programação utilizado neste parte do trabalho é o de memória compartilhada, onde as threads comunicam-se de forma assíncrona através da leitura e escrita em variáveis globais compartilhadas. No contexto do algoritmo K-Means, os vetores de dados (X), centróides (C) e atribuições (assign) representam essa memória compartilhada acessada por múltiplas threads.

O OpenMP suporta programação paralelizada multi-plataforma e com compartilhamento de memória nas linguagens C/C++ e Fortran [OpenMP 2025]. Ele define um modelo portátil e escalável com uma interface simples e flexível para o desenvolvimento de aplicações paralelas em diferentes plataformas.

Utilizamos o OpenMP para paralelizar na CPU as funções de assignment e update da implementação "naive" do algoritmo k-means em 1 dimensão. No caso específico da função assignment, um laço for foi paralelizado. Na função update, optamos por usar acumuladores por thread (`sum_thread[c]`, `cnt_thread[c]`) e reduzir após a região paralela.

Outra opção, não implementada neste trabalho, seria usar `#pragma omp critical` e verificar impactos no desempenho da paralização.

As medições de desempenho consideraram escalonamento em threads $\{1, 2, 4, 8, 16\}$ e o speedup foi calculado como a razão entre o tempo de execução da versão serial do algoritmo e o da implementação em C utilizando os recursos do OpenMP.

Na implementação em C/OpenMP há a possibilidade de fazer um ajuste no processo de schedule (estático vs dinâmico), como também na quantidade de tarefas que será passado para cada thread (chunk size). Além disso, também há a validação de que o SSE não deve aumentar ao longo das iterações.

3.3. Implementação usando CUDA

A segunda etapa do projeto consistiu na portabilidade do algoritmo K-Means para a arquitetura de GPU utilizando a plataforma CUDA. O objetivo principal foi explorar o paralelismo massivo de dados oferecido pelas milhares de threads da GPU para acelerar as etapas computacionalmente intensivas de atribuição (*assignment*) e atualização (*update*).

A implementação seguiu o modelo de computação heterogênea, onde a CPU (Host) gerencia o fluxo de controle e a GPU (Device) executa os cálculos pesados. A estratégia adotada priorizou a minimização da transferência de dados entre Host e Device, reconhecida como um dos principais gargalos em aplicações de GPU.

3.3.1. Gerenciamento de Memória e Transferência de Dados

Diferente da abordagem de memória compartilhada da CPU (OpenMP), em CUDA é necessário gerenciar explicitamente a memória do dispositivo. Foram utilizados `cudaMalloc` para alocação e `cudaMemcpy` para transferência.

Para otimizar o desempenho, a transferência de dados foi estruturada da seguinte forma:

- **Inicialização:** Os vetores de dados (X) e centróides iniciais (C) são copiados para a GPU apenas uma vez, antes do início do laço principal.
- **Durante as Iterações:** Apenas o valor escalar do erro quadrático (SSE) é copiado da GPU para a CPU a cada iteração para verificação do critério de convergência. O vetor de atribuições (*assign*) permanece na memória da GPU, evitando o tráfego desnecessário de N inteiros pelo barramento PCIe a cada ciclo.
- **Finalização:** Apenas após a convergência (ou fim das iterações), os vetores finais de atribuição e centróides são copiados de volta para a CPU.

O algoritmo foi decomposto em três kernels principais (funções `__global__`), desenhados para explorar o paralelismo ao nível de thread: um Kernel de Atribuição (Assignment) e dois Kernels de Atualização (Update).

3.3.2. Kernel de Atribuição (Assignment)

Este kernel é responsável por calcular a distância de cada ponto a todos os centróides. Foi configurado para executar com N threads, onde cada thread é responsável por um único ponto de dado.

- Cada thread calcula a distância Euclidiana do seu ponto para os K centróides, determina o mais próximo e atualiza o vetor `d_assign` na memória global.
- Para o cálculo do SSE total, utilizou-se a função `atomicAdd` para acumular o erro de cada ponto em uma variável global compartilhada (`d_sse`), garantindo a consistência dos dados em um ambiente de escrita concorrente.

3.3.3. Kernels de Atualização (Update)

A etapa de atualização foi implementada seguindo a estratégia de **acumuladores globais na GPU** (Opção B do enunciado). Como múltiplos pontos podem pertencer ao mesmo cluster, a atualização direta das somas geraria condições de corrida. A solução foi dividida em dois passos:

1. **Acumulação (Soma):** Um kernel executado por N threads onde cada thread lê o cluster atribuído ao seu ponto e utiliza operações atômicas (`atomicAdd`) para incrementar a soma das coordenadas (`d_sum`) e o contador de pontos (`d_cnt`) daquele cluster específico.
2. **Cálculo da Média:** Um segundo kernel leve, executado por apenas K threads (uma por cluster), divide a soma acumulada pelo contador para obter a nova posição do centróide.

Vale ressaltar que, para utilizar `atomicAdd` com variáveis de ponto flutuante de dupla precisão (`double`) na GPU Tesla T4 utilizada (arquitetura Turing), foi necessário compilar o código com a flag `-arch=sm_75`.

3.3.4. Medição de Desempenho

Para uma análise precisa, o tempo de execução foi medido utilizando eventos da API CUDA (`cudaEvent_t`), que registram o tempo diretamente na GPU. Isso permitiu segregar o tempo gasto em processamento puro (execução dos kernels) do tempo gasto em latência de transferência de memória (H2D e D2H), permitindo uma análise detalhada do *throughput* computacional.

O *throughput* (vazão) foi calculado considerando o número total de pontos processados ao longo de todas as iterações do algoritmo ($N \times \text{Iterações}$), dividido pelo tempo acumulado de execução dos kernels na GPU. Esta métrica isola a capacidade de processamento da arquitetura, excluindo as latências de transferência de memória (PCIe), e permite visualizar a eficiência computacional. Observou-se que o *throughput* aumentou significativamente com o tamanho do problema, indicando que conjuntos de dados maiores permitem uma melhor amortização dos custos de lançamento de kernel e uma utilização mais eficiente da largura de banda da memória da GPU.

3.4. Implementação usando MPI

Diferente das abordagens anteriores, a implementação com MPI utiliza um modelo de memória distribuída. Neste paradigma, cada processo possui seu próprio espaço de memória privado, e a comunicação e sincronização são realizadas através da troca explícita de mensagens.

4. Resultados

Os resultados obtidos com as implementações paralelizadas do algoritmo k-means 1D seguem abaixo, referentes à escalabilidade forte e escalabilidade fraca.

4.1. Resultados OpenMP

Observou-se que, para os conjuntos de dados Médio e Grande, o algoritmo atingiu o limite configurado de 50 iterações antes de satisfazer o critério de convergência (eps).

Este comportamento é atribuído à distribuição uniforme dos dados gerados aleatoriamente. Diferente de dados com clusters naturais, onde os centróides se estabilizam rapidamente nos centros de densidade, na distribuição uniforme os centróides realizam um processo de ajuste fino prolongado para particionar o espaço de forma equidistante.

Optou-se por manter o limite de 50 iterações para garantir consistência na carga de trabalho computacional entre as versões sequencial e paralela, assegurando que as métricas de speedup reflitam puramente a eficiência da paralelização e não variações no número de iterações.

A tabela 1 mostra o resultado do teste feito com a implementação do algoritmo k-means 1D em C/OpenMP com schedule padrão do compilador (ou seja, sem indicar explicitamente se o schedule deveria ser estático ou dinâmico, nem qual deveria ser o chunk size), tendo como entradas os arquivos de dados correspondente ao conjunto de teste pequeno ($N = 10^4$, $K = 4$).

Tabela 1. Speedup (Escalabilidade Forte) para $N = 10^4$, $K = 4$

#threads	Speedup OpenMP	Tempo Serial (ms)	Tempo OpenMP (ms)
1	0,96	2,40	2,50
2	0,69	2,40	3,50
4	0,52	2,40	4,60
8	0,37	2,40	6,50
16	0,20	2,40	11,90

O gráfico da figura 1 foi feito com base nos dados mostrados na tabela 1.

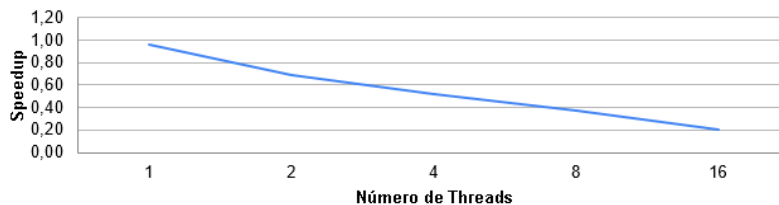


Figura 1. Speedup (Escalabilidade Forte) para $N = 10^4$, $K = 4$

A tabela 2 mostra o resultado do teste feito com a implementação do algoritmo k-means 1D em C/OpenMP com schedule padrão do compilador (ou seja, sem indicar explicitamente se o schedule deveria ser estático ou dinâmico, nem qual deveria ser o chunk size), tendo como entradas os arquivos de dados correspondente ao conjunto de teste médio ($N = 10^5$, $K = 8$).

Tabela 2. Speedup (Escalabilidade Forte) para $N = 10^5$, $K = 8$

#threads	Speedup OpenMP	Tempo Serial (ms)	Tempo OpenMP (ms)
1	1,02	53,80	53,00
2	0,94	53,80	57,00
4	0,94	53,80	57,10
8	0,92	53,80	58,30
16	0,77	53,80	70,30

O gráfico da figura 2 foi feito com base nos dados mostrados na tabela 2.

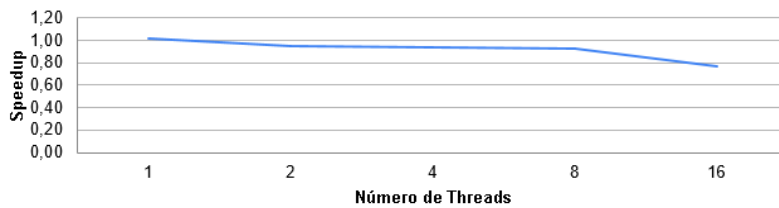


Figura 2. Speedup (Escalabilidade Forte) para $N = 10^5$, $K = 8$

A tabela 3 mostra o resultado do teste feito com a implementação do algoritmo k-means 1D em C/OpenMP com schedule padrão do compilador (ou seja, sem indicar explicitamente se o schedule deveria ser estático ou dinâmico, nem qual deveria ser o chunk size), tendo como entradas os arquivos de dados correspondente ao conjunto de teste médio ($N = 10^6$, $K = 16$).

Tabela 3. Speedup (Escalabilidade Forte) para $N = 10^6$, $K = 16$

#threads	Speedup OpenMP	Tempo Serial (ms)	Tempo OpenMP (ms)
1	1,01	1040,60	1028,10
2	1,11	1040,60	936,40
4	1,13	1040,60	919,50
8	1,14	1040,60	911,70
16	1,12	1040,60	925,60

O gráfico da figura 3 foi feito com base nos dados mostrados na tabela 3.

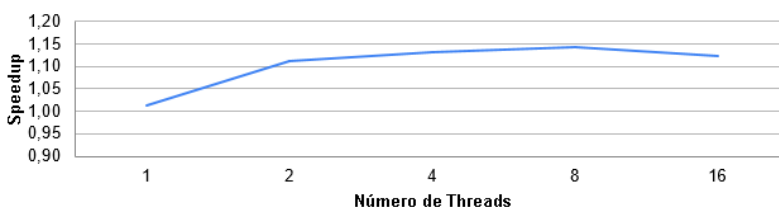


Figura 3. Speedup (Escalabilidade Forte) para $N = 10^6$, $K = 16$

A tabela 4 reorganiza os resultados de testes feitos com a implementação do algoritmo k-means 1D em C/OpenMP com schedule padrão do compilador, registrando o efeito no valor do speedup se aumentarmos o número de threads conforme aumentamos o tamanho do conjunto de teste utilizado.

Tabela 4. Speedup (Escalabilidade Fraca) para $N = 10^6$, $K = 16$

# threads / tamanho conjunto de teste	Speedup OpenMP	Tempo Serial	Tempo OpenMP
4 / pequeno	0,94	53,80	57,10
8 / médio	1,14	1040,60	911,70
16 / grande	1,12	1040,60	925,60

O gráfico da figura 4 foi feito com base nos dados mostrados na tabela 4.

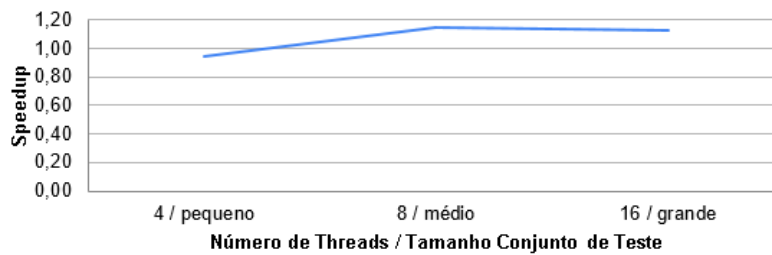


Figura 4. Speedup (Escalabilidade Forte) para $N = 10^6$, $K = 16$

4.2. Análise de Corretude e Reprodutibilidade - OpenMP

A implementação OpenMP demonstrou corretude e reprodutibilidade em todos os testes realizados, atendendo aos critérios de validação estabelecidos.

Primeiramente, o Erro Quadrático (SSE) final, para um mesmo conjunto de dados de entrada, permaneceu consistente em todas as execuções, independentemente do número de threads utilizadas. Isso indica que o algoritmo sempre convergiu para a mesma solução ótima local.

Em segundo lugar, a comparação dos arquivos de saída, realizada pelo script de validação em Python, confirmou que os centróides finais gerados pela versão paralela são numericamente idênticos aos da versão sequencial.

Finalmente, a verificação de convergência monotônica do SSE, implementada dentro do laço iterativo, não acusou nenhum aumento do erro entre as iterações durante a execução dos testes. Isso demonstra que a estratégia de paralelização com acumuladores por thread para a etapa de *update* e a cláusula *reduction* para a etapa de *assignment* foram eficazes em prevenir condições de corrida que pudessem corromper o estado do algoritmo.

4.3. Resultados CUDA

Os testes de desempenho da implementação em GPU foram realizados na plataforma Google Colab, utilizando uma GPU NVIDIA Tesla T4. Foram avaliados três tamanhos de bloco (*Block Size*): 128, 256 e 512 threads.

A Tabela 5 apresenta os tempos totais de execução (incluindo transferências de memória H2D e D2H) e o Speedup calculado em relação à versão sequencial (Serial).

Tabela 5. Desempenho da Implementação CUDA vs. Serial

Dataset	Serial (ms)	Block Size	CUDA Total (ms)	Speedup	Throughput (pt/s)
Pequeno ($N = 10^4, K = 4$)	2,40	128	5,20	0,46x	1,17e8
		256	4,22	0,56x	1,16e8
		512	4,29	0,55x	1,13e8
Médio ($N = 10^5, K = 8$)	53,80	128	27,36	1,96x	1,94e8
		256	27,37	1,96x	1,94e8
		512	27,63	1,94x	1,92e8
Grande ($N = 10^6, K = 16$)	1040,60	128	251,28	4,14x	2,04e8
		256	162,14	6,41x	3,20e8
		512	155,12	6,70x	3,35e8

Os gráficos das figuras 5, 6, 7, 8 foram feitos com base nos dados mostrados na tabela 5.

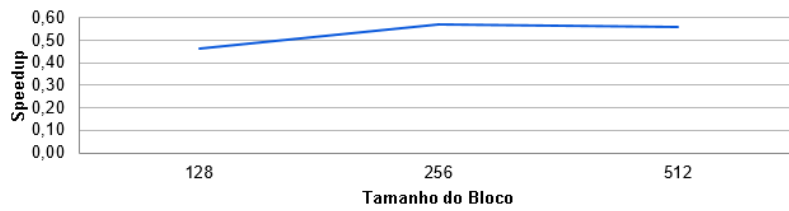


Figura 5. Speedup (Escalabilidade Forte) para $N = 10^4, K = 4$

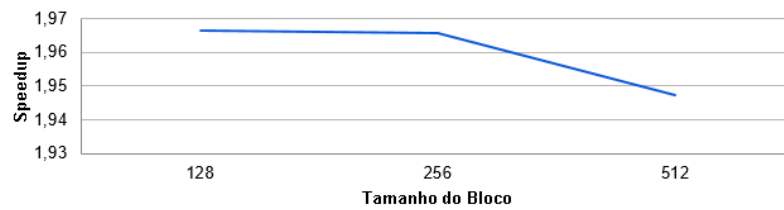


Figura 6. Speedup (Escalabilidade Forte) para $N = 10^5, K = 8$

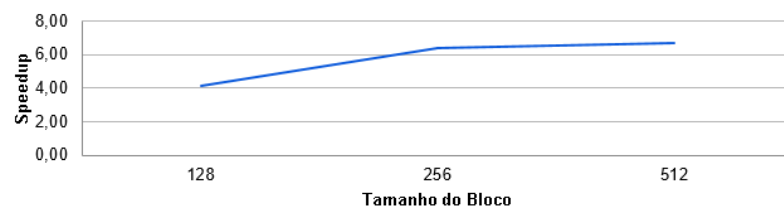


Figura 7. Speedup (Escalabilidade Forte) para $N = 10^6, K = 16$

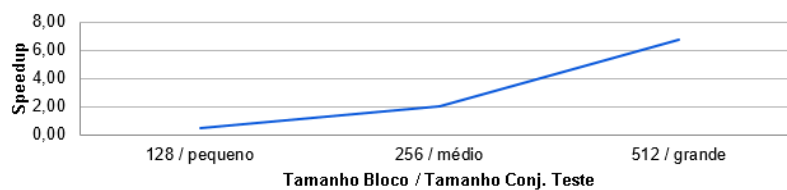


Figura 8. Speedup (Escalabilidade Fraca) - CUDA

4.3.1. Análise de Escalabilidade e Latência

Os resultados evidenciam as características fundamentais da computação em GPU: alta latência inicial e alto *throughput*.

- **Dataset Pequeno (Slowdown):** Para $N = 10^4$, a implementação CUDA foi mais lenta que a sequencial ($\text{Speedup} < 1$). Isso ocorre porque o tempo de execução do kernel é extremamente curto (aprox. 3ms), tornando o custo fixo (*overhead*) de lançamento do kernel e, principalmente, a latência de transferência de dados pelo barramento PCIe (H2D e D2H) dominantes. A GPU não consegue esconder a latência com tão pouco trabalho.
- **Dataset Médio (Transição):** Com $N = 10^5$, o volume de cálculo começa a justificar o uso da GPU. O tempo de processamento dos kernels passa a superar o overhead de transferência, resultando em um speedup de aproximadamente 2x. Nota-se que, para este tamanho, a variação do tamanho do bloco teve impacto irrelevante.
- **Dataset Grande (Alto Desempenho):** O cenário com $N = 10^6$ e $K = 16$ demonstra o verdadeiro potencial da arquitetura. O tempo de execução caiu de $\sim 1040\text{ms}$ (Serial) para $\sim 155\text{ms}$ (CUDA), atingindo um **Speedup de 6,7x**. Neste ponto, a GPU está saturada com trabalho suficiente para manter seus milhares de núcleos ocupados e esconder a latência de acesso à memória global, atingindo um *throughput* de processamento na ordem de $3,3 \times 10^8$ pontos por segundo.

4.3.2. Impacto do Tamanho do Bloco (Block Size)

A escolha do tamanho do bloco influenciou significativamente o desempenho apenas no conjunto de dados Grande.

- Configurações com **256 e 512 threads por bloco** apresentaram desempenho muito superior (tempo de $\sim 155\text{-}160\text{ms}$) em comparação com blocos de 128 threads (tempo de $\sim 251\text{ms}$).
- Isso sugere que blocos maiores permitiram uma melhor **ocupação** (*occupancy*) dos SMs (Streaming Multiprocessors) da Tesla T4 para este kernel específico, ou reduziram a contenção nas operações atômicas globais (`atomicAdd`), visto que blocos maiores resultam em menos blocos totais competindo pela atualização das variáveis globais de soma e contagem.

4.4. Resultados MPI

TBD.

5. Discussão

5.1. Análise do Desempenho com 1 Thread OpenMP

Um resultado notável e consistente em todos os conjuntos de teste foi que a versão OpenMP, mesmo configurada para executar com apenas uma thread, apresentou um speedup superior a 1.0 em relação à versão sequencial "naive". Este fenômeno sugere que o simples ato de compilar com a flag `-fopenmp` pode ter permitido ao compilador GCC aplicar otimizações ao código, ou que a substituição da medição de tempo de `clock()` (tempo de CPU) por `omp_get_wtime()` (tempo de parede) resultou em uma medição mais precisa do tempo real de execução.

5.2. Impacto do Overhead em Cargas de Trabalho Pequenas - OpenMP

Para o conjunto de teste pequeno ($N = 10^4$), o desempenho da paralelização foi negativo, com o speedup caindo drasticamente à medida que o número de threads aumentava (de 1.0 com 2 threads para 0.25 com 8 threads). Este comportamento exemplifica o impacto do overhead de paralelização. Para uma carga de trabalho pequena, o custo computacional de criar, gerenciar e sincronizar as threads é significativamente maior do que o tempo economizado pela divisão do trabalho, resultando que a versão paralela se torna mais lenta que a sequencial.

5.3. Escalabilidade Limitada em Cargas de Trabalho Médias - OpenMP

No conjunto de teste médio ($N = 10^5$), o benefício do paralelismo começa a se manifestar, mas a escalabilidade é limitada. O speedup máximo (próximo de 1.7x) foi obtido com 1 e 2 threads, mas não houve ganho adicional com 4 ou 8 threads, e a performance começou a degradar com 16 threads. Este resultado é compatível com a arquitetura do processador de 2 núcleos físicos e 4 threads lógicas. O ganho inicial reflete o uso dos dois núcleos físicos. O uso de 4 threads (aproveitando o Hyper-Threading) não trouxe melhorias, indicando que, para este tamanho de problema, a sobrecarga de gerenciamento das threads lógicas começa a anular os benefícios do paralelismo adicional.

5.4. Identificação de um Gargalo de Memória em Cargas Grandes - OpenMP

O resultado mais revelador foi observado no conjunto de teste grande ($N = 10^6$). Embora o speedup tenha mostrado uma leve tendência de crescimento com o aumento do número de threads (de 1.03x para 1.20x), o ganho de desempenho geral foi muito baixo. Com 2 núcleos físicos, um speedup teórico ideal seria próximo de 2x, e o valor obtido foi consideravelmente inferior.

Este padrão sugere fortemente que o algoritmo atingiu um gargalo de memória. O algoritmo K-Means, especialmente nas etapas de assignment e update, realiza varreduras intensivas em grandes vetores na memória (X e assign). Em um sistema com memória DDR3 de 4GB e um processador Intel Core i3-5005U, a largura de banda da memória principal, a taxa na qual os dados podem ser lidos ou armazenados em uma memória semicondutora por um processador [Wikipedia contributors 2025] . torna-se o fator limitante (ou, usando uma analogia, a "estrada" entre a memória RAM DDR3 e a CPU é muito estreita e lenta para a demanda do algoritmo). As threads, mesmo disponíveis para processar, passam a maior parte do tempo ociosas, esperando que os dados sejam carregados da RAM para os caches da CPU. Adicionar mais threads não acelera o processo, pois

apenas cria mais "trabalhadores" na fila de espera pelo acesso à memória. Isso explica por que, mesmo com 16 threads, o speedup mal ultrapassou 1.20x. O desempenho não é mais limitado pela capacidade de computação (CPU), mas sim pela velocidade com que os dados podem ser acessados.

5.5. Análise dos resultados CUDA

A análise dos resultados experimentais da implementação em CUDA revela comportamentos distintos de desempenho que corroboram os princípios teóricos da computação massivamente paralela. A discussão a seguir aborda a escalabilidade em função do tamanho do problema e o impacto da configuração de execução na eficiência do hardware.

5.6. Sobrecarga e Escalabilidade - CUDA

Os testes demonstraram que o uso da GPU não é vantajoso para todos os cenários. No conjunto de dados pequeno ($N = 10^4$), observou-se um *slowdown* significativo ($\text{Speedup} < 1$). Este comportamento é explicado pelo **overhead de latência**. O tempo necessário para alocar memória na GPU, transferir os dados via barramento PCIe (Host-to-Device) e lançar os kernels é desproporcionalmente alto comparado ao tempo de execução real do cálculo, que é ínfimo para poucos dados. A GPU, projetada para *throughput* (vazão), sofre quando não há trabalho suficiente para manter suas milhares de unidades de processamento ocupadas.

Por outro lado, a escalabilidade torna-se evidente nos conjuntos médio e grande. O *throughput* computacional saltou de $1,17 \times 10^8$ pontos/s no conjunto pequeno para $3,35 \times 10^8$ pontos/s no conjunto grande. Isso confirma que, à medida que a carga de trabalho aumenta, o custo fixo das transferências e lançamentos é diluído, e o paralelismo massivo da GPU passa a dominar, entregando acelerações de até 6,7x em relação à CPU sequencial.

5.7. Impacto do Tamanho do Bloco e Ocupação dos SMs - CUDA

Um dos achados mais relevantes dos experimentos foi a variação de desempenho entre diferentes tamanhos de bloco (*Block Size*) para o conjunto de dados grande ($N = 10^6$). Enquanto as configurações com 256 e 512 threads por bloco apresentaram tempos similares e ótimos (≈ 155 ms), a configuração com 128 threads foi consideravelmente mais lenta (≈ 251 ms).

Este fenômeno pode ser analisado sob a ótica da **Ocupação** (*Occupancy*) dos Multiprocessadores de Streaming (SMs) da GPU Tesla T4. A ocupação é a razão entre o número de *warps* ativos em um SM e o número máximo teórico que ele pode suportar.

- **Ocultação de Latência:** A arquitetura de GPU depende de uma alta ocupação para esconder a latência de acesso à memória global. Quando um warp fica bloqueado esperando por dados (o que é frequente no K-Means, que varre grandes vetores), o escalonador do SM deve trocar instantaneamente para outro warp pronto para executar.
- **Recursos Limitantes:** A quantidade de blocos que podem residir simultaneamente em um SM é limitada por fatores como registradores e memória compartilhada disponíveis. É possível que, com blocos de 128 threads, o limite de blocos por SM tenha sido atingido antes que o limite de threads ou registradores fosse saturado, resultando em uma subutilização do hardware.

- **Eficiência das Operações Atômicas:** Além disso, o algoritmo utiliza `atomicAdd` em memória global. Com blocos menores (128), o número total de blocos no grid é maior ($GridSize \approx N/128$). Isso pode aumentar a contenção no sistema de memória da GPU, pois mais blocos distintos estão competindo simultaneamente para atualizar as mesmas variáveis globais de soma e contagem.

Portanto, para este kernel específico na arquitetura Turing, blocos de 256 ou 512 threads proporcionaram um equilíbrio melhor entre a disponibilidade de warps para ocultação de latência e a utilização eficiente dos recursos do SM.

6. Conclusão

TBD.

Referências

Ben-Ari, M. (2006). *Principles of Concurrent and Distributed Programming*. Addison-Wesley, 2nd edition.

OpenMP (2025). The openmp api specification for parallel programming. <https://www.openmp.org/>. Acesso em: 14 out. 2025.

Wikipedia contributors (2025). Memory bandwidth — Wikipedia, the free encyclopedia. https://en.wikipedia.org/w/index.php?title=Memory_bandwidth&oldid=1307117362. Acesso em: 15 out. 2025.